

1. A.

$$(a) \text{ Present Value} = \frac{\text{Future Value}}{(1+r)^n}$$

$$V_{0,A}^P = -\$2,533,000 + \frac{\$1,000,000}{1.1^5} + \frac{\$4,000,000}{1.1^{10}} = -\$369,905.52$$

$$V_{0,B}^P = -\$4,000,000 + \frac{\$2,000,000}{1.1^5} + \frac{\$3,000,000}{1.1^{10}} + \frac{\$2,000,000}{1.1^{15}} = -\$1,122,743.38$$

$$(b) \text{ Average annual net benefit} = V_0^P [r(1+r)^{T_p}] / [(1+r)^{T_p} - 1]$$

$$V_A^P = -\$369,905.52 * \frac{[0.1 * 1.1^{10}]}{[1.1^{10} - 1]} = -\$60,200.42$$

$$V_B^P = -\$1,122,743.38 * \frac{[0.1 * 1.1^{15}]}{[1.1^{15} - 1]} = -\$147,611.31$$

(c)

$$V_{0,A}^P = -\$2,533,000 + \frac{\$1,000,000}{1.05^5} + \frac{\$4,000,000}{1.05^{10}} = \$706,179.18$$

$$V_{0,B}^P = -\$4,000,000 + \frac{\$2,000,000}{1.05^5} + \frac{\$3,000,000}{1.05^{10}} + \frac{\$2,000,000}{1.05^{15}} = \$370,826.29$$

(d)

$$V_{0,A}^P = -\$2,533,000 + \frac{\$1,000,000}{1.1^5} + \frac{\$4,000,000}{1.1^{10}} - \frac{\$2,533,000}{1.1^{10}} + \frac{\$1,000,000}{1.1^{15}} + \frac{\$4,000,000}{1.1^{20}} - \frac{\$2,533,000}{1.1^{20}} + \frac{\$1,000,000}{1.1^{25}} + \frac{\$4,000,000}{1.1^{30}} = -\$567,504.21$$

$$V_A^P = -\$567,504.21 * \frac{[0.1 * 1.1^{30}]}{[1.1^{30} - 1]} = -\$60,200.42$$

$$V_{0,B}^P = -\$4,000,000 + \frac{\$2,000,000}{1.1^5} + \frac{\$3,000,000}{1.1^{10}} + \frac{\$2,000,000}{1.1^{15}} - \frac{\$4,000,000}{1.1^{15}} + \frac{\$2,000,000}{1.1^{20}} + \frac{\$3,000,000}{1.1^{25}} + \frac{\$2,000,000}{1.1^{30}} = -\$1,391,519.23$$

$$V_B^P = -\$1,391,519.23 * \frac{[0.1 * 1.1^{30}]}{[1.1^{30} - 1]} = -\$147,611.31$$

B.

$$R(x_1) = 12x_1 - x_1^2$$

$$R(x_2) = 8x_2 - x_2^2$$

$$R(x_3) = 18x_3 - 3x_3^2$$

$$x_1 + x_2 + x_3 \leq 10$$

$$L = (12x_1 - x_1^2) + (8x_2 - x_2^2) + (18x_3 - 3x_3^2) - \lambda(x_1 + x_2 + x_3 - 10)$$

$$\frac{\partial L}{\partial x_1} = 12 - 2x_1 - \lambda = 0$$

$$\frac{\partial L}{\partial x_2} = 8 - 2x_2 - \lambda = 0$$

$$\frac{\partial L}{\partial x_3} = 18 - 6x_3 - \lambda = 0$$

$$\frac{\partial L}{\partial \lambda} = 10 - x_1 - x_2 - x_3 = 0$$

$$x_1 = 10 - x_2 - x_3$$

$$12 - 2x_1 - \lambda = 12 - 2(10 - x_2 - x_3) - \lambda = 2x_2 + 2x_3 - \lambda - 8 = 0$$

$$2x_2 = 8 - \lambda$$

$$2x_2 + 2x_3 - \lambda - 8 = 8 - \lambda + 2x_3 - \lambda - 8 = 0$$

$$x_3 = \lambda$$

$$18 - 6x_3 - \lambda = 18 - 6x_3 - x_3 = 0$$

$$x_3 = \lambda = \frac{18}{7}$$

$$2x_2 = -2x_3 + \lambda + 8 = -x_3 + 8 = -\frac{18}{7} + \frac{56}{7} = \frac{38}{7}$$

$$x_2 = \frac{19}{7}$$

$$x_1 = 10 - x_2 - x_3 = 10 - \frac{19}{7} - \frac{18}{7} = \frac{33}{7}$$

C.

Optimal release plan

	Release 1	Release 2	Release 3
0	10	30	10
10	10, 20	30	10
20	20	40	20
30	30	-	30

I think is easy to solve with final storage volumes as the decision variables because it is possible to find following values with coordinates.

D.

2.

```
1 library(lpSolve)
2
3 f.obj = c(10,8,9)
4
5 f.cons = matrix(c(2,3,1,
6                   5,6,6,
7                   1,1,1,
8                   1,0,0,
9                   0,1,0,
10                  0,0,1), nrow=6, byrow=T)
11
12 f.dir = c("<=",
13           "<=",
14           "<=",
15           ">=",
16           ">=",
17           ">=")
18
19 val = c(1000,
20         2400,
21         700,
22         0,
23         160,
24         0)
25
26 lp1 = lp("max", f.obj, f.cons, f.dir, val)
27
28 lp1$solution
29 |
```

29:1 (Top Level) ↕

Console

Terminal ×

Jobs ×

R 4.1.1 · ~/

```
+           ">=",
+           ">=")
```

```
>
```

```
> val = c(1000,
+         2400,
+         700,
+         0,
+         160,
+         0)
```

```
>
```

```
> lp1 = lp("max", f.obj, f.cons, f.dir, val)
```

```
>
```

```
> lp1$solution
```

```
[1] 240 160 40
```

3.

A. Benefit function

One unit of release = \$2000

Two units of release = \$2000 + \$1500 = \$3500

Three units of release = \$2000 + \$1500 + \$1000 = \$4500

Four units of release = \$2000 + \$1500 + \$1000 + \$500 = \$5000

Five units of release = \$2000 + \$1500 + \$1000 + \$500 - \$1500 = \$3500

Six units of release = \$2000 + \$1500 + \$1000 + \$500 - \$4000 = \$1000

B. Governing equation

Let S_n and FS_n as initial storage and final storage of season n , respectively. Then the governing equation of this problem can be written as

$$FS_n = Q_n + S_n - R_n$$

where Q_n and R_n are the inflow and release of season n , respectively.

Moreover, the final storage of n season is the initial storage of $n + 1$ season. It can be written as

$$FS_n = S_{n+1}$$

C. Backward recursive equation

$$f_n(S_n) = \text{Max}[r_n(S_n, R_n)] \text{ for } n = 4$$

$$f_n(S_n) = \text{Max}[r_n(S_n, R_n) + f_{n+1}(S_{n+1})] \text{ for } n = 1, 2, \text{ and } 3$$

D. Optimal release plan

Storage	Release 1	Release 2	Release 3	Release 4
0	1	1, 2	1	1, 2
1	1, 2	1, 2	1, 2	1, 2
2	1, 2	1, 2	1, 2	1, 2
3	1, 2	1, 2	1, 2	1, 2
4	1, 2	2	1, 2	2

Storage	Final storage 1	Final storage 2	Final storage 3	Final storage 4
0	0	0, 1	0	0, 1
1	0, 1	1, 2	0, 1	1, 2
2	1, 2	2, 3	1, 2	2, 3
3	2, 3	3, 4	2, 3	3, 4
4	3, 4	4	3, 4	4

Code:

```
library(daewr)
```

```
#Storage
```

```
s = c(0, 1, 2, 3, 4)
```

```
#Release
```

```
release.1 = matrix(c(1,0,NA,NA,NA,
                     2,1,0,NA,NA,
                     3,2,1,0,NA,
                     4,3,2,1,0,
                     5,4,3,2,1), nrow=5, byrow=T)
```

```
release.2 = matrix(c(2,1,0,NA,NA,
```

```

3,2,1,0,NA,
4,3,2,1,0,
5,4,3,2,1,
6,5,4,3,2), nrow=5, byrow=T)

```

```

release.3 = matrix(c(1,0,NA,NA,NA,
2,1,0,NA,NA,
3,2,1,0,NA,
4,3,2,1,0,
5,4,3,2,1), nrow=5, byrow=T)

```

```

release.4 = matrix(c(2,1,0,NA,NA,
3,2,1,0,NA,
4,3,2,1,0,
5,4,3,2,1,
6,5,4,3,2), nrow=5, byrow=T)

```

```

release = list(release.1, release.2, release.3, release.4)

```

```

#ft

```

```

benefit.func = c(0,2000,3500,4500,5000,3500,1000)

```

```

ft = list()

```

```

for (i in 1:4){

```

```

  #i=1

```

```

  release.dat = release[[i]]

```

```

  ft.dat = array(NA, c(5,5))

```

```

for (j in 1:5){

  #j=1

  for (k in 1:5){

    #k=1

    r = release.dat[j,k]

    bene = benefit.func[(r+1)]

    ft.dat[j,k] = bene

  }

}

ft[[i]] = ft.dat

}

#Backward recursive process

back.ft =list(ft[[4]], ft[[3]], ft[[2]], ft[[1]])

back.release = list(release.4, release.3, release.2, release.1)

ft.result = list(c(),c(),c(),c())

#First step (T=4)

ft.model = back.ft[[1]]

back.r = back.release[[1]]

```



```

ll = length(ft.result[[1]])

ft.l = list()

release.l = list()

storage.l = list()

for(k in 1:5){

  #k=4

  if (is.na(ft.model[k,1]) & is.na(ft.model[k,2]) & is.na(ft.model[k,3]) & is.na(ft.model[k,4])){

    next

  }

  s0 = max(ft.model[k,],na.rm=T)

  s0.loc = which(ft.model[k,]==s0)

  ss = s[s0.loc]

  r = back.r[k,s0.loc]

  ft.l[[k]] = s0

  release.l[[k]] = c(r)

  storage.l[[k]] = c(ss)

}

ft.r = data.frame(cbind(ft.l), cbind(release.l), cbind(storage.l))

ft.result[[1]][[1]] = ft.r

#Backward recursive algorithm

for (i in 2:100){

  #i=2

```

```
j = mod(i,4)
```

```
bb = mod((i-1),4)
```

```
if (j == 0){
```

```
  j = j+4
```

```
}
```

```
if (bb == 0){
```

```
  bb = bb+4
```

```
}
```

```
ft.model = back.ft[[j]]
```

```
back.r = back.release[[j]]
```

```
ll = length(ft.result[[j]])
```

```
lll = length(ft.result[[bb]])
```

```
back.step.result = ft.result[[bb]][[lll]]
```

```
ft.l = list()
```

```
release.l = list()
```

```
storage.l = list()
```

```
for (k in 1:5){
```

```
  #k=1
```

```
  if (is.na(ft.model[k,1]) & is.na(ft.model[k,2]) & is.na(ft.model[k,3]) & is.na(ft.model[k,4])){
```

```
    ft.l[[k]] = NA
```

```
    release.l[[k]] = NA
```

```
    storage.l[[k]] = NA
```

```

    next
}

for (l in 1:5){

  #l=2

  if (is.na(ft.model[1,l]) & is.na(ft.model[2,l]) & is.na(ft.model[3,l]) & is.na(ft.model[4,l])){

    next

  }

  if (is.na(ft.model[k,l])){

    next

  }

  if (is.na(back.step.result[,1][[l]])){

    ft.model[k,l] = NA

    next

  }

  ft.model[k,l] = ft.model[k,l] + back.step.result[,1][[l]]

}

s0 = max(ft.model[k,],na.rm=T)

s0.loc = which(ft.model[k,]==s0)

ss = s[s0.loc]

r = back.r[k,s0.loc]

ft.l[[k]] = s0

release.l[[k]] = c(r)

storage.l[[k]] = c(ss)

```

```
}
```

```
ft.r = data.frame(cbind(ft.l), cbind(release.l), cbind(storage.l))
```

```
ft.result[[j]][[(ll+1)]] = ft.r
```

```
if (ll>4){
```

```
  a = unlist(ft.result[[j]][[(ll-1)]][,1]) - unlist(ft.result[[j]][[ll]][,1])
```

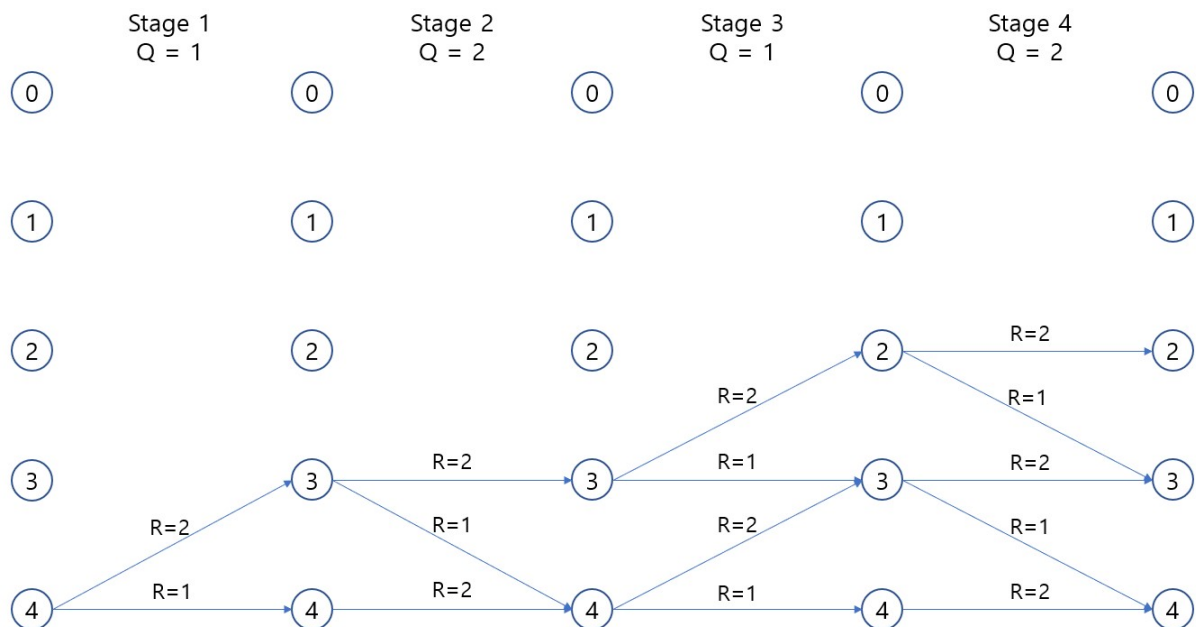
```
  b = unlist(ft.result[[j]][[ll]][,1]) - unlist(ft.result[[j]][[(ll+1)]][,1])
```

```
  if (a[1]==b[1] & a[2]==b[2] & a[3]==b[3] & a[4]==b[4] & a[5]==b[5]) break
```

```
}
```

```
}
```

E.



4.

B.

```
path = 'C:/Users/Lee/Desktop/2022년1학기/수자원시스템공학/HW2/'
```

```
#####
```

```
f.release.rule = read.csv(paste0(path,'full_release_rule.csv'),header = T)
```

```
inflow = read.csv(paste0(path,'inflow.csv'),header = T)
```

```
colnames(inflow) = c('date','inflow')
```

```
demand = read.csv(paste0(path,'demand.csv'),header = T)
```

```
#####
```

```
initial.storage = 40.025
```

```
Smax = 108.7
```

```
Smin = 6.2
```

```
flexible.demand = (3 * 3600 * 24) / 1000000
```

```
#####
```

```
storage = array(NA, c(365))
```

```
storage[1] = inflow[1,2] - demand[1,2] + initial.storage
```

```
for (i in 2:365){
```

```
  #i=282
```

```
  release.rule = f.release.rule[i,2:5]
```

```
  inflows = inflow[i,2]
```

```
  demands = demand[i,2:4]
```

```
  stor = storage[(i-1)]
```

```
  if (stor > release.rule[1]){
```

```
    dd = flexible.demand + sum(demands[2:3])
```

```
    storage[i] = max(Smin, inflows + stor - dd)
```

```

}else if (release.rule[1] > stor & stor > release.rule[2]){
  storage[i] = max(Smin, inflows + stor - sum(demands[1:3]))
}else if (release.rule[2] > stor & stor > release.rule[3]){
  storage[i] = max(Smin, inflows + stor - sum(demands[1:2]))
}else if (release.rule[3] > stor & stor > release.rule[4]){
  if (demand[i,1]==6){
    storage[i] = max(Smin, inflows + stor - sum(demands[1]) - (0.8 * sum(demands[2])))
  }else if (demand[i,1]==7 | demand[i,1]==8 | demand[i,1]==9){
    storage[i] = max(Smin, inflows + stor - sum(demands[1]) - (0.7 * sum(demands[2])))
  }else{
    storage[i] = max(Smin, inflows + stor - sum(demands[1:2]))
  }
}else{
  storage[i] = max(Smin, inflows + stor - (0.8 * sum(demands[1:2])))
}
}

```

```
act.release = array(NA, c(365))
```

```
act.release[1] = sum(demand[1,2:4])
```

```
for (i in 2:365){
```

```
  act.release[i] = storage[(i-1)] + inflow[i,2] - storage[i]
```

```
}
```

```
LHS = initial.storage + sum(inflow[,2])
```

```
RHS = storage[365] + sum(act.release)
```

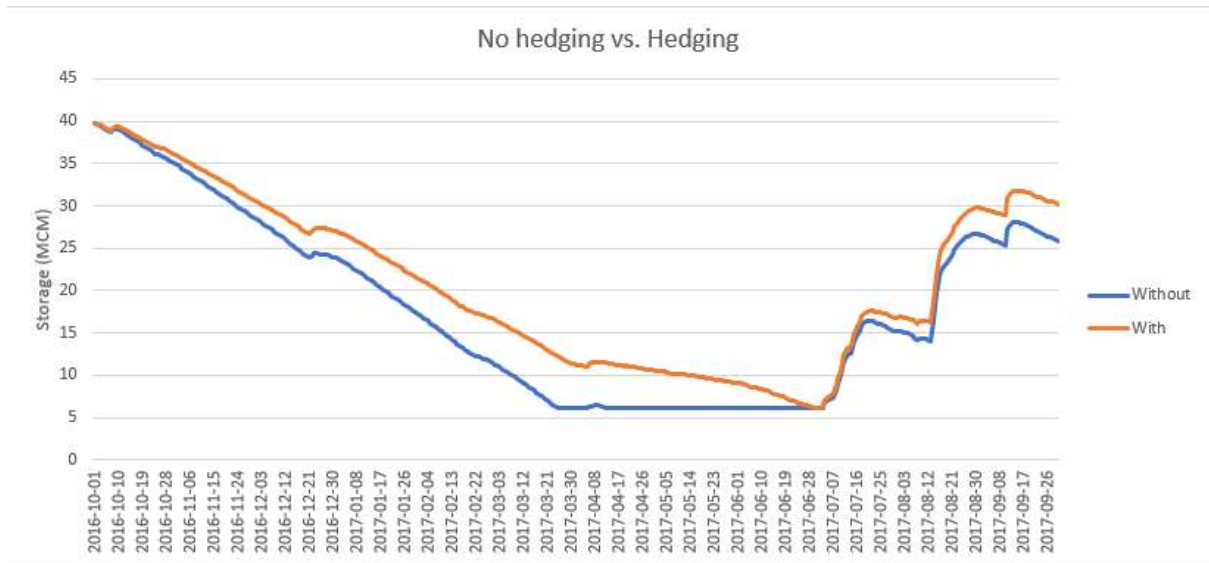
```
write.csv(storage, file=paste0(path,'hedging_storage.csv'), row.names = F)
```

C.

LHS = 120.8349

RHS = 120.8695

D.



Because the effect of hedging rules, the duration of the minimum storage decreased. Moreover, considering hedging rule, the ability to recover from drought seems much better than when hedging rule is not considered.